

# Chapter 10 - Tile and Sprite Layers from BASIC

The six picture chips each have their own way of arranging the screen out of small repeatable elements. This chapter pulls those elements together into a single summary and shows how to drive them from BASIC. Use it as a recipe book; each chip's own chapter holds the full details.

## 10.1 The two basic ideas

**Tiles.** A tile is a small rectangular bitmap (typically  $8 \times 8$ ) that the chip repeats across a grid. Every cell of the grid names one tile and a colour pair. The video chip looks up each tile's bitmap and draws it. Tiles save memory: an  $80 \times 25$  text screen needs only 2,000 bytes of cell data plus one shared character set, instead of  $80 \times 25 \times 64 = 128,000$  pixels of bitmap.

**Sprites.** A sprite is a small bitmap that the chip can place at any pixel position, independently of the tile grid. Sprites are drawn on top of the tile picture, usually with one colour reserved as transparent. They are the chip's hardware support for moving objects (a missile, a player, a cursor) that should not disturb the rest of the picture.

The chips offer different mixes of these two ideas:

| Chip       | Tile model                                                | Sprite model                         |
|------------|-----------------------------------------------------------|--------------------------------------|
| VideoChip  | None (bitmap framebuffer)                                 | None (use the blitter)               |
| VGA        | $80 \times 25$ character tiles in text mode               | None                                 |
| TED video  | $40 \times 25$ character tiles, character ROM/RAM         | None                                 |
| ANTIC+GTIA | $40 \times 24$ character tiles + bitmap modes             | 4 players (8 px) + 4 missiles (2 px) |
| ULA        | $32 \times 24$ character cells, attribute-coloured bitmap | None                                 |
| Voodoo 3D  | None (triangle pipeline)                                  | Textured quads (two triangles)       |

VideoChip and Voodoo have no dedicated tile/sprite engine, so this chapter focuses on the four character-based chips and on Voodoo's quad-based sprites.

## 10.2 Tile layers from BASIC

### 10.2.1 VGA text mode

The simplest tile layer is the VGA text mode (SCREEN 3). Each character cell is a 2-byte pair in the text region at 0xB8000: character code, then attribute. The attribute's low nibble is the foreground colour (0–15); the next three bits are the background colour (0–7); the top bit is blink.

```
10 SCREEN 3                : REM 80x25 text mode
20 COLOR 14, 1              : REM yellow on blue
30 LOCATE 10, 30
40 PRINT "HELLO, WORLD"
```

To poke a character directly without going through PRINT, write the cell at  $0xB8000 + (\text{row} * 80 + \text{col}) * 2$ . Chapter 5 has the full description.

## 10.2.2 TED text mode

TED ON followed by TED MODE 0 selects the C16/Plus-4 character mode. The video matrix is a  $40 \times 25$  array of character codes; each cell's colour byte lives in the parallel colour RAM at offset  $0x400$  from the matrix base. TED CLS clears the matrix with spaces; TED CHAR and TED VIDEO select the character set and matrix base.

```
10 TED ON
20 TED MODE 0 : REM standard text
30 TED COLOR 6, 14 : REM blue background, dark blue border
40 TED CLS
50 REM (POKE characters into the matrix here)
```

Multi-colour TED text is selected with TED MODE 2; the four background colours BG\_COLOR0–3 become the four available colours per cell, with the top two bits of each character byte selecting per-cell foreground.

## 10.2.3 ANTIC character modes

ANTIC uses a **display list** to switch tile mode line by line. The standard 40-column text mode is mode 2. Set ANTIC CHBASE to the high byte of your character set, build a display list whose first entry sets LMS to point at the video matrix, and write the list address to ANTIC DLIST.

```
10 ANTIC ON
20 ANTIC MODE &H22 : REM enable display list DMA + normal width
30 ANTIC DLIST &H6000 : REM display list at $6000
40 ANTIC CHBASE &HE0 : REM character set at $E000
```

The Atari mode list reaches from mode 2 (40 columns) through mode 7 (20 columns, double-size). Chapter 7 lists every mode.

## 10.2.4 ULA attribute-coloured bitmap

The ULA's "tiles" are a  $32 \times 24$  grid of  $8 \times 8$  cells over a shared bitmap. Each cell's attribute byte sets the INK and PAPER colours for any pixel of the bitmap that falls inside the cell. Writing a character to the screen is therefore a two-step process: plot the bitmap, then write the attribute.

```
10 ULA ON
20 ULA CLS &H38 : REM white paper, black ink everywhere
30 REM plot pixels with ULA PLOT
40 ULA ATTR 5, 5, &H02 : REM cell (5, 5) becomes red ink
```

## 10.3 Sprite layers from BASIC

### 10.3.1 ANTIC/GTIA players and missiles

ANTIC and GTIA together provide four 8-pixel **players** and four 2-pixel **missiles**. Each can be moved independently in X with GTIA PLAYER and GTIA MISSILE. The graphics bytes are either DMA-fetched from the player-missile area (set with ANTIC PMBASE and enabled with GRCTL) or written directly into GTIA GRAFP/GRAFM. Each player has its own colour register (COLPM0–3).

```

10 ANTIC ON
20 ANTIC PMBASE &HF0           : REM PM data at $F000
30 GTIA GRCTL 3                 : REM enable player + missile graphics
40 GTIA COLOR 5, &H46           : REM player 0 colour
50 GTIA PLAYER 0, 100, 1       : REM player 0 to x = 100, double-size
60 GTIA GRAFP 0, &H81           : REM 8-pixel bit pattern for player 0

```

GTIA PRIOR controls priority: which players appear in front of which playfield colour, whether missiles act as a fifth player, and which GTIA-only colour mode is in use.

### 10.3.2 Voodoo textured quads

Voodoo has no dedicated sprite engine, but a textured quad (two triangles sharing a diagonal) is the standard substitute. Upload the sprite bitmap as a texture, then draw a screen-aligned quad at the sprite's position. The same pipeline handles rotation, scaling, and alpha blending for free.

A flat-shaded "sprite" is the easiest case. The fragment below draws a  $32 \times 32$  red square at (100, 100) as two triangles sharing a diagonal:

```

10 VOODOO ON
20 VOODOO DIM 640, 480
30 VOODOO TRICOLOR 255, 0, 0   : REM flat red at vertex A
40 REM lower-left triangle
50 VERTEX A 100, 100
60 VERTEX B 132, 100
70 VERTEX C 100, 132
80 TRIANGLE
90 REM upper-right triangle
100 VERTEX A 132, 100
110 VERTEX B 132, 132
120 VERTEX C 100, 132
130 TRIANGLE
140 VOODOO SWAP

```

For a textured sprite, upload the sprite bitmap with TEXTURE BASE/DIM/UPLOAD/ON, then write the texture coordinate start values and per-pixel gradients with VOODOO TRIUV before each triangle. The TRIUV form is **not** per-vertex; it takes one start value pair and the four dS/dX, dT/dX, dS/dY, dT/dY gradients in 14.18 fixed-point. Chapter 9 §9.4.2 lists every register the texture pipeline uses and the fixed-point conventions.

Use VOODOO CHROMAKEY ON and a chroma colour to make one texel colour transparent - the equivalent of a sprite mask.

## 10.4 Combining layers

Because the compositor stacks the six chips automatically (see Chapter 3), you can use one chip for tiles and another for sprites. A few patterns that work well:

- **VGA text overlay on Voodoo.** Run Voodoo at  $640 \times 480$  underneath; use VGA text mode as a heads-up status line on top. Layers 10 and 20 cleanly separate the two.
- **TED tiles with ANTIC players.** TED draws the playfield on layer 12; ANTIC/GTIA places the moving objects on layer 13. Where the player-missile colour register is non-zero, the player is drawn; everywhere else the TED picture shows through.
- **ULA bitmap with ULA attribute changes.** Within a single chip: write the bitmap once, then animate the attributes for cheap colour-cycling tricks.

A program that needs more than one of these may safely turn chips on simultaneously; they share neither registers nor VRAM and cannot interfere.

## 10.5 What comes next

Part II of this manual ends here. Part III covers the audio side: nine sound engines that feed a single mixer in much the same way the six picture chips feed the compositor.